

EEE5390 Final Project

Group 3

Members:

David Merveilleux

Ayush Shashikant Pindoria

Shanny May Ruiz

Grigoriy Zolotarev

Purpose of Project

The final project focused on completing an ASIC implementation flow for a RISC-V SoC. The project was a collaborative initiative divided into two connected stages.

Task 1 used Synopsys to synthesize RTL into gate-level netlist and those outputs were directly inputted into Task 2 where we performed physical design, including floorplanning, power planning, placement, clock-tree synthesis, routing, verification checks and final design export.

A major specific issue we had to focus on throughout the project was the SRAM integration. The SoC was not only a standard cell design, but it also included SRAM macro instances used by the cache/memory. These macros had to be handled consistently across both tasks. During synthesis, the compiler needed the SRAM Verilog models and timing database files so the modules could be resolved and linked. During physical design, ICC2 needed the SRAM physical abstracts such as the .lef and ndm views so that the macros could be placed and connected in the floorplan.

Our group's objective was not just to run individual TCL scripts, but to build a working synthesis to layout flow for the RISC-V SoC.

Task 1 Overview

The goal of Task 1 was to synthesize the RISC-V SoC and this step converted the RTL level verilog description of the system into gate level netlist mapped to the SkyWater 130 nm cell library. The generated netlist, SDC constraints, and synthesis reports were required outputs for Task 1 and became the starting point for the physical design flow in Task 2.

We used the synthesis flow from Lab 5 where we the rv32imsu RISC-V core was synthesized instead. The final project expanded the scope from RISC-V core by itself to a complete RISC-V SoC.

We also had to verify that the design met timing before physical implementation and check if there were any unresolved modules, missing libraries or incorrect restraints. This is because any timing violations would carry into Task 2's floorplanning, placement, CTS and routing.

Task 1 Implementation and Issues

The member in charge of Task 1 focused on connecting the proper library files that the TA provided for the SRAM modules. The library used the SkyWater 130 nm and then the link library was expanded to include both the standard cell library and the SRAM macro timing libraries. This allowed the compiler to accurately recognize the timing and power behavior of the SRAM macros. The largest technical issue for the SRAM macro instances was at first the SRAMs were being treated as unresolved references because the compiler could not actually connect and view

the SRAM. The solution to this was to check the creation of our libraries and including the proper .db timing files in the link library. This was important because this same library linking was used in Task 2. We also tried our best to use relative paths so it would be easier for the code to be portable.

Timing constraints was another challenge where successful synthesis needed the proper clock period. If the clock period was too aggressive, the compiler would report a negative slack. The files pulled from the timing report and QoR command were used to check worst negative slack, total negative slack, and the number of violating paths. We ended up choosing the period 45 which got rid of all our timing violations.

Task 1 Conclusion

The biggest lesson from Task 1 was that synthesis depends heavily on the correct setup. The RTL files, top modules, libraries and constraints all have to match the project structure. A small mistake could easily cause unresolved modules, incorrect synthesis or unusable outputs for physical designs.

Task 1 Report (from qor_report)

Metric	Final Value
Clock period	45
Worst negative slack (WNS)	0
Total negative slack (TNS)	0
Number of violating paths	0
Cell area	2951037.891450
Leakage power	3.4559e+05 nW
Total power	15.0674 mW

Task 2 Overview

The goal of Task 2 was to take the synthesized RISC-V SoC output from Task 1 and complete the physical design implementation using Synopsys Compiler. This part of the project followed the same general flow as Lab 7, where the physical design process includes floorplanning, placement, clock tree synthesis and routing.

Task 2 was more difficult than Task 1 because the design could no longer be checked only as a logical netlist; the design now had to satisfy physical layout requirements such as macro placement, chip boundaries, power/ground connectivity, routing congestion, DRC cleanliness, and post route timing.

Task 2 Implementation and Issues

The first major part of Task 2 was setting up the ICC2 environment. This included creating the design library, loading the SkyWater 130 nm technology information, reading the netlist and applying the SDC constraint file generated from Task 2.

A major difficulty in this setup stage was the SRAM macro integration. In Task 1, the SRAM macros required Verilog and .db timing views so the Design Compiler could resolve the SRAM references. In Task 2, ICC2 also needed the physical views of the macros. This meant the SRAM macros needed to be visible through .lef or .ndm physical library views so that they could be placed as hard macros instead of being treated like missing or abstract logic. This caused a lot of debugging because the physical design flow depended on both standard cell physical libraries and SRAM macro physical libraries being loaded correctly. If the macro physical views were missing or named incorrectly, ICC2 could not floorplan, place, or connect the SoC correctly.

The next major step was floorplanning. The final project required the floorplan to define the physical chip area, utilization, aspect ratio, and power distribution network. The Lab 7 flow explains that floorplanning includes design/library setup, design constraints, power planning, floorplan definition, and pin placement. Our group used this structure but had to modify it for the SoC because the SoC included SRAM macros. Unlike a standard-cell-only design, the SRAM macros had fixed physical sizes and needed to be manually placed.

Macro placement became one of the first major physical design problems. In early attempts, some SRAM macros were overlapping each other or were placed too close to the chip boundary. This created placement and routing problems later in the flow. The fix was to manually control the SRAM macro coordinates and check macro locations. The best way our group solved these issues was the use the gui interface to physically see where our SRAM modules were on the board and how the spacing problems were affecting the design.

This step showed an important part of physical design tradeoff. If the SRAM macros were spaced too far apart, the critical paths between the processor logic and the macros became too long and would hurt the setup timing. If the SRAM macros were placed too close together, routing congestion and DRC violations increased. Our final placement strategy was a balanced compact layout where the SRAMs were close enough to reduce long timing paths, but not so close that the router had no channel space.

Power Distribution Network

After the floorplan and macro locations were established, we created the power distribution network. The project required power rings, standard cell rails, and a power mesh to connect the design's VDD and VSS supplies.

In our design, this was more complicated because the standard cells and SRAM macros did not use the exact same pin naming style. The standard cells used power/ground pins such as

VPWR/VGND while the macros used supply pins such as vccd1 etc. Therefore, the PDN portion of the TCL script had to connect the logical VDD and VSS nets to both the standard cell pins and the SRAM macro pins.

This caused several debugging issues, and it was around floorplanning and this point that the group decided to all run their independent versions of Task 2 to try and get the best result. Some early runs showed floating power or ground shapes, floating macro connections or missing PG connectivity. These were not simple syntax errors; they were physical connectivity problems caused by the power network not being properly stitched to all the standard cells and macros. We verified through the connectivity reports whether the macro pins were being found and adjusted the connected_pg_net commands based on our findings. This stage was important because a design can appear placed and routed but still be invalid if cells or SRAMs are not actually connected to the power and ground.

Placement Optimization

After floorplanning and PDN setup, placement was performed using ICC2 placement optimization. Placement automatically placed standard cells around the fixed SRAM macros. According to the Lab 7 flow, placement is the stage where the standard cells are placed automatically, and after place_opt the design should be checked for congestion, utilization, timing, and area.

This step showed that macro placement strongly affected the entire rest of the design. In early runs, poor macro arrangement created extremely bad timing and routing congestion. One reported early timing result had setup slack around -60ns, which showed that the physical paths were much too long or poorly optimized. The placement also created routing pressure when macros blocked too much routing area.

To improve the result, the macro locations were adjusted to reduce long critical paths while keeping enough routing space between macros. We also used timing driven placement so that ICC2 would consider timing during standard cell placement. The goal was not only to make the layout legal, but also to give the optimizer a better physical starting point before CTS and routing.

Clock Tree Synthesis

After placement, CTS was needed because the clock signal drives many sequential elements, so it cannot be treated like a normal data signal. Lab 7 explains that CTS builds and optimizes the clock network to balance skew, latency, and slew.

In our design, CTS was important because the RISC-V SoC contained many sequential cells and a high-fanout clock network. If the clock tree was not built properly, hold violations, clock skew,

and transition violations could appear after placement. We ran CTS and then used timing and clock reports to inspect the clock network quality.

The clock optimization stage helped improve timing but did not completely eliminate setup timing difficulty. This was expected because CTS can improve clock distribution and reduce skew-related issues, but it cannot fully compensate for a bad floorplan or long macro to logic paths. Therefore, the biggest timing improvements came from a combination of better macro placement, placement optimization, CTS, and routing optimization.

Routing and DRC Fixing

Routing was the final major implementation step which physically connected all standard-cell pins, SRAM macro pins, block ports, and clock/data nets using metal layers. Lab 7 describes `route)opt` as a command that routes interconnects, checks timing, fixes timing, modifies routing, and fixes DRC violations iteratively.

Early routing attempts produced many DRC violations. These violations were mainly caused by poor macro placement, tight routing channels and congestion around the SRAM macros. The fix was not only to run DRC repair commands, but to improve the physical design before routing. We adjusted macro spacing, used a cleaner floorplan and allowed enough routing resources between hard macros. After this, routing became much cleaner. The final reported result from the team was 0 DRC violations, which showed the final macro arrangement and routing flow were physically valid.

Timing Closure Challenge

Timing was the main challenge of Task 2. In task 1, the design was synthesized using a relaxed 45 ns clock period and achieved 0 WNS, 0 TNS, 0 violating paths. However, after physical implementation, the timing became harder because real placement, routing, wire delay, clock insertion delay and macro locations affected the path delays.

The largest setup timing problem appeared when the macros were placed poorly or too far from the related logic. In one early result, setup WNS was extremely negative, around -60 ns. This showed that the physical layout had created long routes and poor timing paths. The team improved this through several changes:

- Reducing macro distance from related logic
- Using a more compact but still routable floorplan,
- Running timing driven placement
- Checking timing reports after each stage
- Compare wns/tns after optimization changes.

The final reported setup WNS was approximately -0.48 ns. This means the design was very close to setup timing closure, although not completely clean. Hold timing was reported as nearly clean.

This result showed major improvement compared with the initial very negative timing and confirmed that physical layout decisions were the dominant factor in the timing quality.

The most important lesson was that timing and DRC were competing constraints. A very spread-out macro layout reduced routing congestion but created long timing paths. A very compact macro layout improved timing but created congestion and DRC risk. The final design used a balanced compact layout, which produced clean DRC and much better setup timing.

LVS and Extraction Debugging

After routing, we also worked on layout verification and extraction. This stage was challenging because the routed design had to be exported and checked outside the normal ICC2 optimization flow. The LVS/debug process exposed issues related to extracted netlists, SRAM macro handling, pin matching, and top-level connectivity.

One difficulty was that the SRAM GDS/LEF/macro views had warnings during extraction, including repeated cell placement warnings inside SRAM macro structures. Another difficulty was top-level pin matching during LVS, where the extracted layout and schematic/netlist did not always match cleanly at first. These issues were separate from ordinary ICC2 route DRC and showed that a design can pass route DRC but still require careful debugging for LVS and extracted connectivity.

Due to time limits and extraction/pin-matching issues, we were not able to generate a clean final LVS report for submission, so our final verification evidence focused mainly on ICC2 routing DRC, timing reports, routed design exports, and the final layout files.

Team Challenges and Workflow Issues

This project was also difficult because the work was hard to distribute between the members. Since the workflow is very linear (step-by-step) and each step is affected by results previously run, it was hard to pass around, or jump inbetween members work to run certain commands. Instead we ended up distributing Task 2 as a tag team of efficiency where different members were attempting to improve timing and setup violations at different times and they would pass their best trials to the next person available based on our schedule.

Another smaller challenge was that different people had different TCL files and different local paths so some scripts that used personal absolute paths made it harder to reproduce each other's results. We had to compare working and non-working scripts and modify the flow so that it matched our own project directory structure.

Task 2 Conclusion

The biggest lesson from Task 2 was that physical design depends on both correct tool setup and good physical judgement. The design could not be completed by only running the original Lab 7 script because the final project SoC contained SRAM hard macros. These macros required special handling in library setup, floorplanning, macro placement, PG connection, routing and LVS/export checks.

The most important technical challenge was balancing macro placement, timing and routability. When macros were too far apart, timing became very poor.

Although setup timing was still slightly negative at approximately -0.48 ns, the design improved significantly from early runs with very large negative slack. The final routed design achieved 0 routing DRC violations and was close to timing closure. Overall, Task 2 showed how the synthesized netlist from Task 1 is transformed into a real physical layout and how physical efforts such as wire delay, congestion, macro placement, and clock routing determine final design quality.

Overall, this project showed the full connection between RTL synthesis and physical implementation for a RISC-V SoC. In Task 1, we learned how important the setup stage is because the RTL files, SRAM models, timing libraries, constraints, and top module all had to be correctly linked before synthesis could produce usable outputs. In Task 2, we saw that even a correctly synthesized design can become difficult once real physical effects are introduced. The SRAM macros created the largest challenge because they affected library setup, macro placement, power connectivity, routing congestion, timing, and verification. Our team had to debug unresolved macro references, physical library issues, PG connectivity, DRC violations, and setup timing problems through repeated report checking and script changes. The final result was a routed and optimized RISC-V SoC implementation with clean routing DRC and timing that was brought close to closure. Although the project was challenging, especially with team coordination and long tool runtimes, it gave us a practical understanding of how ASIC design depends on both correct automation scripts and careful engineering decisions at each stage of the flow.